# Chapter 4 Solutions

## CH04-Q01

The load path is $30 + 200 + 120 + 180 + 250 + 20 + 40 = 840$ ps. This is a simplified lower bound before adverse skew, jitter, and omitted wires. Hold timing requires a separate minimum-delay check.

The stated arithmetic bypass saves $250 + 20 = 270$ ps, giving 570 ps. Improving that noncritical path by another 100 ps leaves the common clock at 840 ps. Reducing the critical load path by 80 ps instead permits 760 ps if no other path becomes critical. Every instruction then benefits from the shorter common period, at the implementation cost of faster memory.

## CH04-Q02

| Instruction | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| I1 | 1 | 2 | 3 | 4 | 5 |
| I2 | 2 | 3 | 4 | 5 | 6 |
| I3 | 3 | 4 | 5 | 6 | 7 |
| I4 | 4 | 5 | 6 | 7 | 8 |
| I5 | 5 | 6 | 7 | 8 | 9 |
| I6 | 6 | 7 | 8 | 9 | 10 |
| I7 | 7 | 8 | 9 | 10 | 11 |

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Completion takes 11 cycles. Finite-run average IPC is $7/11 \approx 0.636$, whereas steady-state IPC is one. The stated 200 ps pipeline period gives 2200 ps; seven unpipelined instructions at 1000 ps each take 7000 ps. Elapsed-time speedup is $7000/2200 = 35/11 \approx 3.1818$. Dividing cycle counts alone would ignore their different durations.

At 200 ps per stage, latency is $5(200) = 1000$ ps and total time is $11(200) = 2200$ ps. Steady-state completion interval is 200 ps. If an unpipelined instruction instead takes the sum of five equal stage delays, its seven-instruction time is 7000 ps and the time speedup is $7000/2200 \approx 3.18$.

# Chapter 4 Solutions (continued)

## CH04-Q03

| Instruction | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| I1 | 1 | 2 | 3 | 4 | 5 |
| I2 | 2 | 3 | 4 | 5 | 6 |
| I3 | 3 | 4 | 5 | 6 | 7 |

I2's EX in cycle 4 uses I1's EX/MEM result for `$8`. I3's EX in cycle 5 uses I1's MEM/WB result for `$8` and I2's EX/MEM result for `$11`.

Without the corresponding bypass, I2 can use the old register-file `$8`. I3 can use an old `$8` or an incorrect `$11` computed from stale input. These are architectural errors, not merely longer execution.

## CH04-Q04

The producer has `MemRead=1`, destination `$2`, and the waiting AND reads `$2`. Hold PC and IF/ID, insert a zero-side-effect bubble into ID/EX, and allow the load to advance.

| Instruction | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| I1 load | 1 | 2 | 3 | 4 | 5 |
| I2 AND | 2 | 3, 4 held | 5 | 6 | 7 |
| Bubble | | | 4 | 5 | 6 |
| I3 OR | 3, 4 fetch held | 5 | 6 | 7 | 8 |

The load's MEM/WB data reaches I2's EX in cycle 5. I2 cannot occupy EX in cycle 4 simultaneously with the bubble.

A store's `rt` is a real data source. However, store data is ultimately needed in MEM rather than for address addition in EX. A design with a suitable MEM-stage store-data bypass can avoid the stall for this load-to-store-data dependence. A design requiring the data in EX must stall. The base-register dependence is a different case. The hardware model must state its paths.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Chapter 4 Solutions (continued)

## CH04-Q05

| Step | Prediction | Outcome | New state | Result |
|---|---|---|---|---|
| 1 | N | T | WT | Miss |
| 2 | T | T | ST | Hit |
| 3 | T | N | WT | Miss |
| 4 | T | T | ST | Hit |
| 5 | T | N | WT | Miss |
| 6 | T | N | WN | Miss |
| 7 | N | N | SN | Hit |

There are four misses and accuracy $3/7 \approx 42.9\%$. The specified recovery costs $4(2) = 8$ wrong-path slots. Predicting taken when the outcome is not taken also fetches the wrong path.

Accuracy omits resolution depth, target prediction, BTB misses, aliasing, branch types, and recovery latency. Those determine the actual cycle cost.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## CH04-Q06

Retain I1's committed effects and let I2 and I3 finish normally. The handler must see the architectural state after I3 and before I4. Prevent ordinary register and memory writes by I4, I5, I6, and later instructions.

At T, save I4's PC, overflow cause, required bad address, and privilege/context state while I2 commits. At T+1, let I3 finish WB/commit. This controller redirects at T+2, after the handler-visible state includes all effects through I3. This chapter's model has no architectural delay slot. Appendix B's BD/EPC rule is a separate model.

In a ROB implementation, completed younger work stays uncommitted. When I4 reaches the head after all older instructions commit, discard I4 and younger entries and restore the appropriate mappings. This separates execution completion from architectural commitment.

## CH04-Q07

```
stall = ID_EX.MemRead and ID_EX.rt != 0 and
        ((IF_ID.uses_rs and ID_EX.rt == IF_ID.rs) or
         (IF_ID.uses_rt and ID_EX.rt == IF_ID.rt))
```

# Chapter 4 Solutions (continued)

This condition assumes the marked sources need the loaded value at the next EX stage. Later-use store data can use a more specific timing-aware condition.

`$zero` writes have no architectural destination effect. Treating them as dependencies creates needless stalls. A load uses `rs` for its base but writes `rt`; `uses_rt` prevents an equal destination index from masquerading as a source dependence.

When both valid sources match, use the newest produced value, normally EX/MEM over MEM/WB. An unavailable load result must not be replaced with the address in EX/MEM.

## CH04-Q08

The percentages have cycles as their denominator.

$$IPC = 0.60(4) + 0.30(2) + 0.10(1) = 3.10.$$

Approximate cycles are $10000/3.10 = 3225.81$. A finite execution uses integral cycles and may only approximate the given proportions. Time is

$$\frac{3225.81}{2 \times 10^9} = 1.61290\ \mu\text{s}.$$

The two-wide alternative needs 5000 cycles and $5000/(2.4 \times 10^9) = 2.08333\ \mu\text{s}$. The four-wide model is faster by $2.08333/1.61290 \approx 1.29167$.

Using $0.6(10000)/4$ would incorrectly treat a cycle fraction as an instruction fraction. Unmodeled prediction, cache, functional-unit, port, and power effects need additional evidence or an updated observed issue distribution.

## CH04-Q09

$$CPI = 0.45(4) + 0.20(3) + 0.20(5) + 0.15(4) = 4.0,$$
$$T = 10^9(4)(300\ \text{ps}) = 1.2\ \text{s}.$$

Single-cycle time is $10^9(1)(800\ \text{ps}) = 0.8$ s, so it wins under these inputs. More cycles can outweigh a shorter period. For equality,

$$10^9(4)t = 0.8\ \text{s} \quad \Rightarrow \quad t = 200\ \text{ps}.$$

Any larger multicycle period loses with unchanged CPI/mix.

## CH04-Q10

$$mispredict_cycles = 0.20(0.60)(2) = 0.24, \qquad CPI = 1 + 0.24 + 0.10 = 1.34.$$

# Chapter 4 Solutions (continued)

In a no-slot ISA both younger fetched instructions are wrong-path. In classic MIPS the first is required architectural work and only the later one is flushed. Dynamic IC also includes that executed slot; replacing penalty two with one alone compares different execution counts.

At T, capture I4's PC/cause/address, allow I2 to commit, and block I4/I5/I6 writes. At T+1, allow I3 to commit while the exception remains buffered. At T+2, redirect to the handler using saved context. Its state equals sequential completion through I3, with no later register or memory effects.

## CH04-Q11

`0x02324021` has op 0, rs 17, rt 18, rd 8, funct `0x21`: `addu t0,s1,s2`. RegDst 1, ALUSrc 0, MemtoReg 0, RegWrite 1 select 7+5=12 into register 8. No memory write occurs.

`0x8e09000c` has op `0x23`, rs 16, rt 9, imm 12: `lw t1,12(s0)`. RegDst 0, ALUSrc 1, MemtoReg 1, RegWrite 1, MemRead 1 select ALU inputs 0x1000 and 12, yielding address `0x100c`. Memory value `0xabcdef01` enters t1.

`0xae0afffc` has op `0x2b`, rs 16, rt 10, imm `0xfffc`=-4: `sw t2,-4(s0)`. ALUSrc 1, MemWrite 1, RegWrite 0 yield address `0x0ffc` and store RtData `0x12345678`. RegDst/MemtoReg cannot affect commit. All three independently advance PC to `0x00400024`.

`0x1232fffe` is `beq s1,s2,-2`. PC4=0x00400024, displacement=-8, BT=0x0040001c. With 7−5=2, Zero 0 selects PC4. With s1=5, Zero 1 and Branch 1 select BT. Neither case writes registers/data memory.

At the region boundary PC4=0x10000000. Concatenating its high nibble 1, 26 zero bits, and 00 gives JT=0x10000000. Using the old PC nibble produces 0 and the wrong region.

ALUSrc 0 incorrectly uses RtData in address arithmetic:

$$0x1000 + 0x12345678 = 0x12346678.$$

This is an address-input mux error. Correct address `0x0ffc` with a wrongly connected immediate data wire stores `0xfffffffc` instead of `0x12345678`. ALUSrc must not replace the independent Reg[rt]→memory-data branch.

# Chapter 4 Solutions (continued)

## CH04-Q12

With forwarding (cycles; repeated entries mark held stages):

| Instruction | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| I1 | 1 | 2 | 3 | 4 | 5 |
| I2 | 2 | 3 | 4 | 5 | 6 |
| I3 | 3 | 4, 5 | 6 | 7 | 8 |
| I4 | 4, 5 | 6 | 7 | 8 | 9 |

C4's load-use check makes C5.EX a bubble while I1 and I2 advance. C5.WB writes `$2=0x33`, so ID/EX can latch that older value at C5's end. In C6, MEM/WB forwards `0x3c`, overriding it and producing `$4=0x3c AND 0x0f=0x0c`. In C7, EX/MEM forwards `0x0c`, producing `$8=0x0c OR 0x80=0x8c`. Completion is at C9.

Without forwarding:

| Instruction | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| I1 | 1 | 2 | 3 | 4 | 5 |
| I2 | 2 | 3 | 4 | 5 | 6 |
| I3 | 3 | 4, 5, 6 | 7 | 8 | 9 |
| I4 | 4, 5, 6 | 7, 8, 9 | 10 | 11 | 12 |

Lin Cing https://github.com/xinhi6716/computer-science-self-study

I3 waits for I2's C6 write, then reads in that cycle's second half. I4 similarly waits for I3's C9 write. The two waits add two cycles each: `4+4+2+2=12`. Final values remain `$2=0x3c`, `$4=0x0c`, and `$8=0x8c`; timing changes, program results do not.

In part 4, I3 no longer reads `$2`, so no load-use stall is needed. I3.EX in C5 computes `$4=0x55 AND 0x0f=0x05`; I4.EX in C6 obtains `0x05` from EX/MEM and produces `$8=0x85`. Four instructions without stalls finish WB at C8, taking eight cycles. I1 and I2 share a destination but in-order write-back preserves the correct final value; a shared destination alone is not a RAW hazard.

# Chapter 4 Solutions (continued)

## Verification code appendix: hazard_timeline.py

```
"""Cycle-level five-stage schedules for ALU forwarding and load-use stalls."""

from dataclasses import dataclass


@dataclass(frozen=True)
class Instr:
    name: str
    dest: str | None = None
    sources: tuple[str, ...] = ()
    load: bool = False
    stages: tuple[int, ...] = ()


def dependent(previous: Instr, current: Instr) -> bool:
    return previous.dest not in (None, "$zero", "$0") and previous.dest in current.sources


def schedule_pair(previous: Instr, current: Instr, mode: str) -> tuple[Instr, Instr]:
    """Return two five-stage rows under an explicitly named hazard policy."""
    producer = Instr(previous.name, previous.dest, previous.sources, previous.load, (1, 2, 3,
4, 5))
    if not dependent(previous, current):
        consumer_stages = (2, 3, 4, 5, 6)
    elif mode == "forward":
        consumer_stages = (2, 3, 4, 5, 6)
    elif mode == "no-forward":
        consumer_stages = (2, (3, 4, 5), 6, 7, 8)
    elif mode == "load-use":
        consumer_stages = (2, (3, 4), 5, 6, 7)
    else:
        raise ValueError(f"unknown mode: {mode}")
    return producer, Instr(current.name, current.dest, current.sources, current.load,
consumer_stages)


def display(row: Instr) -> str:
    cells = []
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Chapter 4 Solutions (continued)

```
    for stage in row.stages:
        cells.append(str(stage) if isinstance(stage, int) else "/".join(map(str, stage)))
    return f"{row.name}: IF {cells[0]}, ID {cells[1]}, EX {cells[2]}, MEM {cells[3]}, WB
{cells[4]}"


def main() -> None:
    producer_alu = Instr("I1 add $8,$9,$10", "$8", ("$9", "$10"))
    consumer_alu = Instr("I2 sub $11,$8,$12", "$11", ("$8", "$12"))
    producer_load = Instr("I1 lw $2,20($1)", "$2", ("$1",), load=True)
    consumer_load = Instr("I2 and $4,$2,$5", "$4", ("$2", "$5"))
    independent = Instr("I2 or $6,$1,$7", "$6", ("$1", "$7"))

    cases = (
        ("ALU with forwarding", schedule_pair(producer_alu, consumer_alu, "forward")),
        ("ALU without forwarding", schedule_pair(producer_alu, consumer_alu, "no-forward")),
        ("load-use with forwarding", schedule_pair(producer_load, consumer_load, "load-
use")),
        ("load then independent", schedule_pair(producer_load, independent, "forward")),
    )
    for title, rows in cases:
        print(title)
        for row in rows:
            print(" ", display(row))

    assert schedule_pair(producer_alu, consumer_alu, "forward")[1].stages == (2, 3, 4, 5, 6)
    assert schedule_pair(producer_alu, consumer_alu, "no-forward")[1].stages == (2, (3, 4,
5), 6, 7, 8)
    assert schedule_pair(producer_load, consumer_load, "load-use")[1].stages == (2, (3, 4),
5, 6, 7)
    assert schedule_pair(producer_load, independent, "forward")[1].stages == (2, 3, 4, 5, 6)
    assert not dependent(Instr("discard", "$zero", load=True), Instr("read zero", sources=
("$zero",)))
    print("pipeline checks: PASS")


if __name__ == "__main__":
    main()
```

# Chapter 4 Solutions (continued)

## Verification code appendix: single_cycle_model.py

```
"""One-cycle word-addressed MIPS subset without delay slots or exception handling."""
from __future__ import annotations

MASK=(1<<32)-1

def step(word: int, pc: int, registers: list[int], memory: dict[int,int]):
    if len(registers)!=32 or registers[0]!=0 or pc&3:
        raise ValueError('aligned PC and 32 registers with r0=0 required')
    op=word>>26;rs=(word>>21)&31;rt=(word>>16)&31;rd=(word>>11)&31
    imm=word&65535; signed_imm=imm if imm<32768 else imm-65536
    a,b=registers[rs],registers[rt];pc4=(pc+4)&MASK

control=dict(RegDst=None,ALUSrc=0,MemtoReg=None,RegWrite=0,MemRead=0,MemWrite=0,Branch=0,Jump
=0,ALUOp=None)
    new_regs=registers.copy();new_mem=memory.copy()
    if op==0:
        control.update(RegDst=1,ALUSrc=0,MemtoReg=0,RegWrite=1,ALUOp=2)
        fun=word&63
        if fun==0x21:alu=(a+b)&MASK
        elif fun==0x23:alu=(a-b)&MASK
        elif fun==0x24:alu=a&b
        elif fun==0x25:alu=a|b
        elif fun==0x2a:
            signed=lambda x:x if x<(1<<31) else x-(1<<32)
            alu=int(signed(a)<signed(b))
        else:raise ValueError('unsupported R-type function')
    elif op in (0x23,0x2b):
        control.update(ALUSrc=1,ALUOp=0)
        alu=(a+signed_imm)&MASK
        if alu&3:raise ValueError('word memory requires aligned address')
        if op==0x23:control.update(RegDst=0,MemtoReg=1,RegWrite=1,MemRead=1)
        else:control.update(MemWrite=1)
    elif op==4:
        control.update(Branch=1,ALUOp=1)
        alu=(a-b)&MASK
    elif op==2:
        control.update(Jump=1)
```

# Chapter 4 Solutions (continued)

```
        alu=0
    else:raise ValueError('unsupported opcode')
    data=memory[alu] if control['MemRead'] else None
    write_data=data if control['MemtoReg']==1 else alu if control['MemtoReg']==0 else None
    dest=rd if control['RegDst']==1 else rt if control['RegDst']==0 else None
    if control['MemWrite']:new_mem[alu]=b
    if control['RegWrite'] and dest!=0:new_regs[dest]=write_data
    zero=alu==0
    bt=(pc4+(signed_imm<<2))&MASK
    jt=(pc4&0xf0000000)|((word&0x3ffffff)<<2)
    next_pc=jt if control['Jump'] else bt if control['Branch'] and zero else pc4
    trace=dict(control=control,rs=rs,rt=rt,rd=rd,ReadA=a,RtData=b,ALU_B=signed_imm&MASK if
control['ALUSrc'] else
b,ALUResult=alu,WriteReg=dest,WriteData=write_data,Zero=zero,PC4=pc4,BT=bt,JT=jt,nextPC=next_
pc)
    return next_pc,new_regs,new_mem,trace

def main():
    regs=[0]*32;regs[16]=0x1000;regs[17]=7;regs[18]=5;regs[10]=0x12345678
    memory={0x100c:0xabcdef01}
    for word in [0x02324021,0x8e09000c,0xae0afffc,0x1232fffe,0x08100400]:
        pc,out_regs,out_mem,trace=step(word,0x00400020,regs,memory)
        print(f'{word:08x}: nextPC={pc:08x}, ALU={trace["ALUResult"]:08x}, WriteReg=
{trace["WriteReg"]}, WriteData={trace["WriteData"]}')
    assert step(0x02324021,0x400020,regs,memory)[1][8]==12
    assert step(0x8e09000c,0x400020,regs,memory)[1][9]==0xabcdef01
    assert step(0xae0afffc,0x400020,regs,memory)[2][0xffc]==0x12345678
    assert step(0x1232fffe,0x400020,regs,memory)[0]==0x400024
    equal=regs.copy();equal[17]=5
    assert step(0x1232fffe,0x400020,equal,memory)[0]==0x40001c
    assert step(0x08000000,0x0fffffc,regs,memory)[0]==0x10000000
    assert regs[8]==0 and 0xffc not in memory
    assert step(0x02320021,0x400020,regs,memory)[1][0]==0
    print('single-cycle state effects: PASS')

if __name__=='__main__':main()
```

### References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study